

Boss Clash Ball

1. Project Overview

An action 2D game where the player faces off with the boss using ball. Players can use various abilities and dashing mechanics to avoid attacks and deflect the ball from the boss to itself. The game combines simple controls with progressively more difficult boss battles.

2. Project Review

A relevant existing project is boss battle game, where players face off against bosses with unique abilities. These games often have basic movement and combat systems.

Improvements:

- **Dash and Ability Skills:** Player abilities and dash skills with a cooldown to make the player manage the movement and adds a strategic layer to the gameplay. Players will need to master when to dash to avoid attacks or use their special ability to deal significant damage.
- **Dynamic Boss Phases:** The bosses will have unique phases that change their attack patterns based on health thresholds, increasing challenge and excitement.

3. Programming Development

3.1 Game Concept

Game Concept:

Objective:

- The player must defeat a series of progressively stronger bosses by using their ball, abilities, and dash skills while avoiding boss attacks.

Mechanics:

- The player can move, use abilities, and dash across the screen using simple controls (arrow keys for movement, spacebar for deflect, a dash key for quick movement and a unique skill).
 - Each boss features multiple phases that change their behavior and attacks based on their remaining health.

3.2 Object-Oriented Programming Implementation

CharacterSelection Class:

- Role: Manages the character selection screen, allowing the player to pick a character, enter their name, and view stats or quit the game.
- Attributes: characters, selected_character, player_name, quit_button, view_stats_button, button_font
- Methods:
 - draw_title(): Displays the game title at the top of the screen.
 - draw_button(): Draws "Quit Game" and "View Stats" buttons.
 - get_player_name(): Prompts the player to enter their name using a dialog box.
 - draw_instructions(): Renders instructions for selecting a character.
 - draw_characters (): Draws character selection boxes and labels them with numbers.

- `handle_input()`: Processes user inputs for character selection and game quitting.

DataCollector Class:

- Role: Handles game data collection and logging for analytics such as dash usage, ability usage, playtime, boss defeats, and character selection.
- Attributes: `data`, `dash_file`, `ability_usage_file`, `playtime_file`, `boss_defeated_file`, `character_choice_file`
- Methods:
 - `init_csv(file_name, headers)`: Creates a new CSV file with headers if it doesn't exist or prepares it for appending.
 - `record_dash_usage(boss_type, dash_count)`: Logs dash usage data.
 - `record_ability_usage(character, count)`: Logs ability usage data.
 - `record_playtime(playtime)`: Logs total playtime.
 - `record_boss_defeated(character_id, defeated)`: Logs whether a boss was defeated by a character.
 - `record_character_choice(character)`: Logs selected character.
 - `add_csv(file, row)`: Appends a new row to the specified CSV file.

StatsViewer Class:

- Role: Manages the display and interaction with game statistics, including ability usage, boss defeats, character choices, dash usage, and playtime.
- Attributes: button_font, title_font, ability_button, boss_button, character_button, dash_button, playtime_button, back_button, buttons, data_collector, character_names, root
- Methods:
 - `_init_fonts()`: Initializes the fonts for button text and titles.
 - `_init_buttons()`: Defines button positions and creates button rectangles.
 - `draw()`: Draws the main menu with buttons and title on the screen.
 - `handle_input()`: Handles user input and button clicks to navigate to different statistics pages.
 - `cleanup()`: Cleans up resources, closes graphs, and closes the Tkinter window.
 - `show_ability_graph()`: Starts a thread to display the ability usage graph.
 - `show_defeat_graph()`: Starts a thread to display the boss defeat graph.
 - `show_character_choice()`: Starts a thread to display the character choice graph.
 - `show_dash_graph()`: Starts a thread to display the dash usage graph.
 - `show_time_play()`: Starts a thread to display the playtime graph.
 - `create_playtime_graph()`: Generates and displays the playtime box plot.
 - `create_dash_usage_graph()`: Generates and displays the average dash usage graph for fire and ice bosses.
 - `create_character_choice_graph()`: Generates and displays a pie chart for character selection statistics.
 - `create_defeat_graph()`: Generates and displays a bar chart for boss defeat statistics by character.

- `create_ability_graph()`: Generates and displays a bar chart for ability usage statistics by character.
- `run()`: Main game loop to handle user interaction and display statistics.

Player Class:

- **Role:** Manages the player's movement, deflect, ability usage, and dash mechanics.
- **Attributes:** `x`, `y` (position), `color`, `name`, `hp`, `moving`, `deflect_range`, `transparent`, `invisibility_end_time`, `wall`, `boss_type`, `dashes`, `frozen`, `speed_reduction`, `image`, `image_width`, `image_height`, `deflect_particles`, `hitbox_width`, `hitbox_height`, `hitbox_x`, `hitbox_y`, `start_time`, `total_playtime`, `ability_usage_count`, `boss_defeated`
- **Methods:**
 - `draw()`: Draws the player on the screen.
 - `get_hitbox()`: Returns the hitbox for collision detection.
 - `take_damage(amount)`: Reduces player health when hit.
 - `dash()`: Moves the player quickly in the last movement direction if the cooldown allows.
 - `use_ability(ball)`: Uses the player's special ability (freeze, wall, invisibility).
 - `deflect(ball, boss_x, boss_y)`: Deflects the boss's attack towards the boss.
 - `update_movement(keys)`: Updates the player's movement based on key inputs.
 - `update()`: Updates the player's status (e.g., handling invisibility).

DeflectParticle Class:

- Role: Draws the effect of deflect skill
- Attributes: x, y (position), size, color, speed_x, speed_y, lifetime, birth_time
- Methods:
 - update(): Updates the particle's position and size based on its speed and lifetime.
 - draw(): Draws the particle on the screen.

Wall Class:

- **Role:** Represents a wall that the Blue character can summon, which interacts with the ball.
- Attributes: x, y, width, height, speed, active
- Methods:
 - move(): Moves the wall.
 - draw(): Renders the wall to the screen.
 - check_collision(ball): Checks if the ball collides with the wall

Boss Class:

- Role: Handles the boss's health, behavior, and special abilities.
- Attributes: x, y (position), width, height, hp, fireball_cooldown, lava_time, snowball_cooldown, fireballs, lavas, snow_patches, snowballs
- Methods:
 - `fireball_barrage(player)`: Launches a barrage of fireballs at the player if the cooldown period has passed.
 - `update_fireballs(player)`: Updates and draws the fireballs, making them move towards the player and interact with them.
 - `create_lava(player)`: Creates a lava patch at a random position within a specified range from the player, if no active lava patch exists.
 - `update_lavas(player)`: Updates the lava's state, checks its collision with the player, and damages the player if they are standing on the lava.
 - `snowball(player)`: Launches snowballs towards the player if the cooldown time has passed.
 - `update_snowballs(player)`: Updates the position of the snowballs and checks for collisions with the player.
 - `create_snow(player)`: Creates snow patches around the boss, ensuring they don't overlap with each other or the player.
 - `draw()`: Draws the boss on the screen.

Lava Class:

- Role: Represents a lava patch created by the boss, which can damage the player if they stand on it.
- Attributes: x, y (position), width, height, duration, time_created, active
- Methods:
 - draw(): Draws the lava patch on the screen with a color.
 - check_collision(player): Checks if the player collides with the lava patch, returning True if they do.
 - update(): Deactivates the lava after the specified duration has passed.

Fireball Class:

- Role: Represents a fireball fired by the boss that moves towards the player and deals damage on collision.
- Attributes: x, y (position), radius, speed, active, direction_x, direction_y
- Methods:
 - draw(): Draws the fireball on the screen.
 - move_towards_player(player): Moves the fireball towards the player's position and checks for out-of-bound conditions.
 - update(player): Updates the fireball's position, moving it towards the player and checking for collisions. If it collides with the player, the player takes damage and the fireball is deactivated.

Snowball Class:

- Role: Represents a snowball fired by the boss that moves towards the player and deals damage on collision.
- Attributes: x, y (position), radius, speed, active, direction_x, direction_y
- Methods:
 - draw(): Draws the snowball on the screen using its current position.
 - move_towards_player(player): Moves the snowball towards the player's position based on the calculated direction, and deactivates if it moves out of bounds.
 - update(player): Updates the snowball's position, moving it towards the player and checking for collisions. If it collides with the player, the player takes damage, and the snowball is deactivated.

Snow Class:

- Role: Represents snow patches on the ground. If the player steps on it, they are slowed down or frozen.
- Attributes: x, y (position), width, height, active, player_on_snow_time, player_was_on_snow
- Methods:
 - draw(): Draws the snow patch on the screen using its position and size.
 - check_collision(player): Checks if the player collides with the snow patch. If there's a collision, the player's speed is reduced, and they might get frozen after a set time.
 - update(): Updates the snow patch's state, deactivating it after the set duration.

Game Class:

- Role: Manages the entire game flow, including the player, boss, health bars, timer, and game over conditions. Handles input and updates the game state.
- Attributes: character_selection, player, ball, data_collector, boss_type, boss_num, boss, game_start_time, game_time, timer_font, game_over_screen, boss_health_bar, player_health_bar, boss_defeated
- Methods:
 - update_health_bars(): Updates the health bars for both the player and the boss.
 - draw_health_bars(): Draws the health bars for both the player and the boss on the screen.
 - initialize_boss(): Initializes a new boss with a random type.
 - format_time(seconds): Formats the time into a MM:SS format for display.
 - draw_timer(): Draws the timer on the screen, showing the elapsed game time.
 - start_game(): Starts the main game loop, handling input, drawing the screen, updating the player and boss, and checking for game over conditions.

AnimatedHealthBar Class:

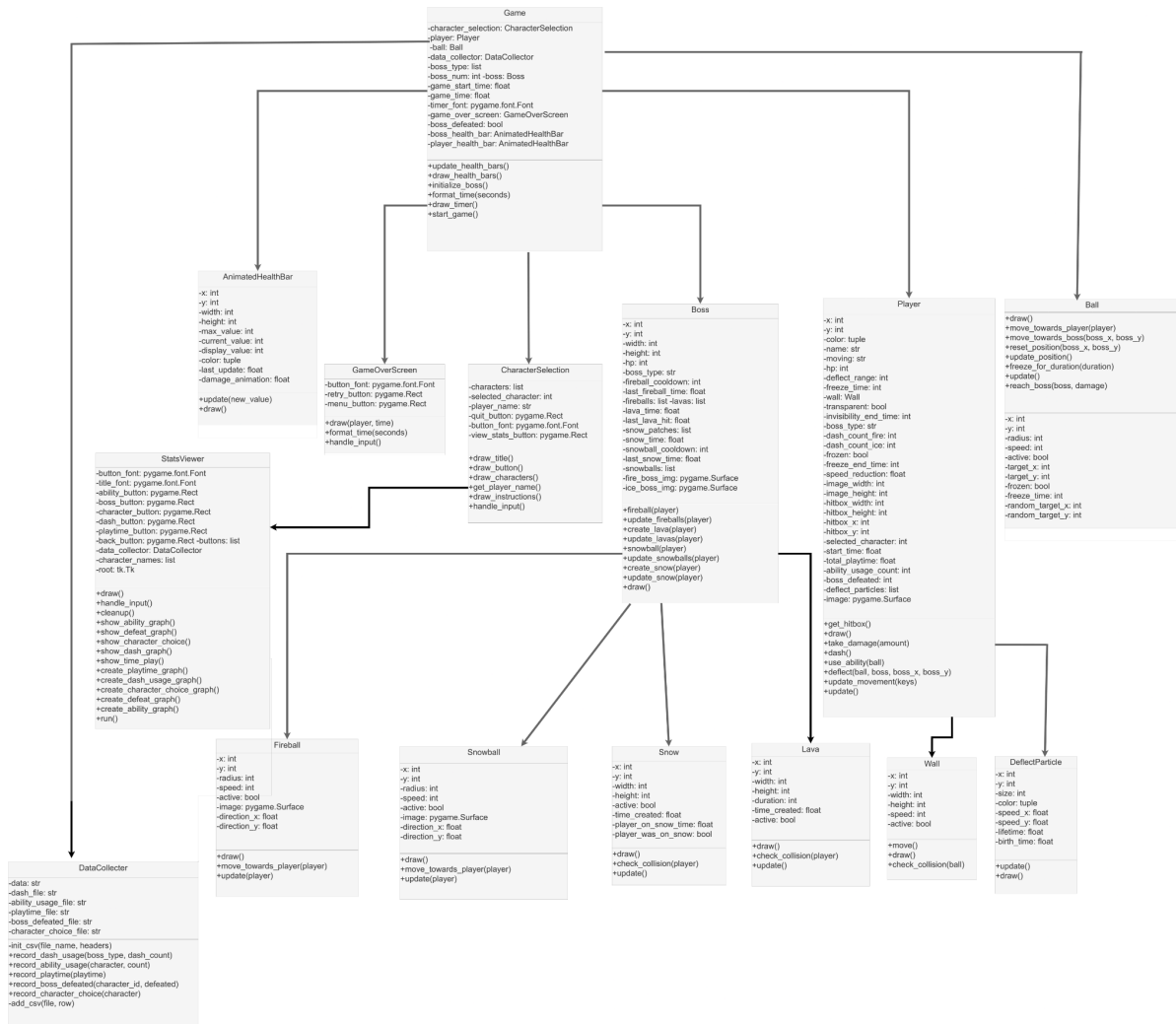
- Role: Represents a health bar with animation, showing the player's or boss's health status.
- Attributes: x, y, width, height, max_value, current_value, display_value, color, last_update, damage_animation
- Methods:
 - update(new_value): Updates the current health value and animates the change in the health bar, including the damage animation.
 - draw(): Draws the health bar on the screen, showing the current health and animation effects (e.g., flash when taking damage).

GameOverScreen Class:

- Role: Displays the game over screen, including buttons to retry or return to the main menu, and shows relevant game statistics like time survived and bosses defeated.
- Attributes: button_font, retry_button, menu_button
- Methods:
 - draw(player, time): Draws the game over screen with overlays, statistics like time survived, and number of bosses defeated. Also draws the retry and menu buttons.
 - format_time(seconds): Formats the elapsed time into a string format
 - handle_input(): Handles input for the game over screen, checking for button clicks (retry or menu).

Ball Class:

- Role: Represents the projectile that the player deflected the ball.
- Attributes: x, y (position), radius, speed, active, target_x, target_y, frozen, freeze_time, random_target_x, random_target_y
- Methods:
 - `draw()`: Draws the ball on the screen.
 - `move_towards_player(player)`: Moves the ball towards the player.
 - `move_towards_boss(boss_x, boss_y)`: Moves the ball toward the boss when deflected.
 - `update_position()`: Updates the ball's movement towards its target.
 - `reset_position(boss_x, boss_y)`: Respawns the ball at the boss's location.
 - `freeze_for_duration(duration)`: Freezes the ball for a specific duration.
 - `update()`: Checks if the freeze time has expired and updates the ball's behavior.
 - `reach_boss(boss, damage=False)`: Handles interactions with the boss (either deflecting or damaging).



if you not see clearly you can download this from google drive:

 [uml_diagram.png](#)

3.3 Algorithms Involved

Movement Logic: Detects and updates player and ball positions based on keyboard input.

Collision Detection: Checks if the ball collides with the boss to deal damage.

Event-driven Mechanics: Actions like ability usage and movement are driven by player inputs (key presses) or random events (like the boss using abilities).

4. Statistical Data (Prop Stats)

4.1 Data Features

Average Player Dash Usage: Track dash usage per game

Average Ability Usage: Track ability usage per character

Most Chosen Character by Player: Track character selection

Time Played: Measures how long the player has been in each session.

Enemies Defeated: Tracks the number of bosses defeated by the player.

3.2 Data Recording Method

Data will be stored in a CSV file. Each row will contain metrics such as time played, enemies defeated.

3.3 Data Analysis Report

The data will be presented in tables for easy visualization.

Feature	Why It's Good to Have	How to Obtain 50 Values	Class & Variable	How to Display
Average Player Dash Usage Per Boss	To balance dash ability and see player behavior.	Track dash usage per game session for 50 sessions.	Player class, dash_count	bar chart
Average Ability Usage (Per Character)	Helps with character and ability balance.	Track ability usage per character over 50 sessions.	Player class, ability_usage_count	Bar chart
Time Played Per Player	Understand engagement and game length preferences.	Track total time played for 50 sessions.	Player class, total_playtime	boxplot
Enemies Defeated	Tracks how often the boss is defeated by player	Tracks how often the boss is defeated by a player in 50 games.	Player class, boss_defeated	bar chart
Most Chosen Character by Player	Understand player preferences for characters.	Track character selection across 50 sessions.	Player class, selected_character	pie chart

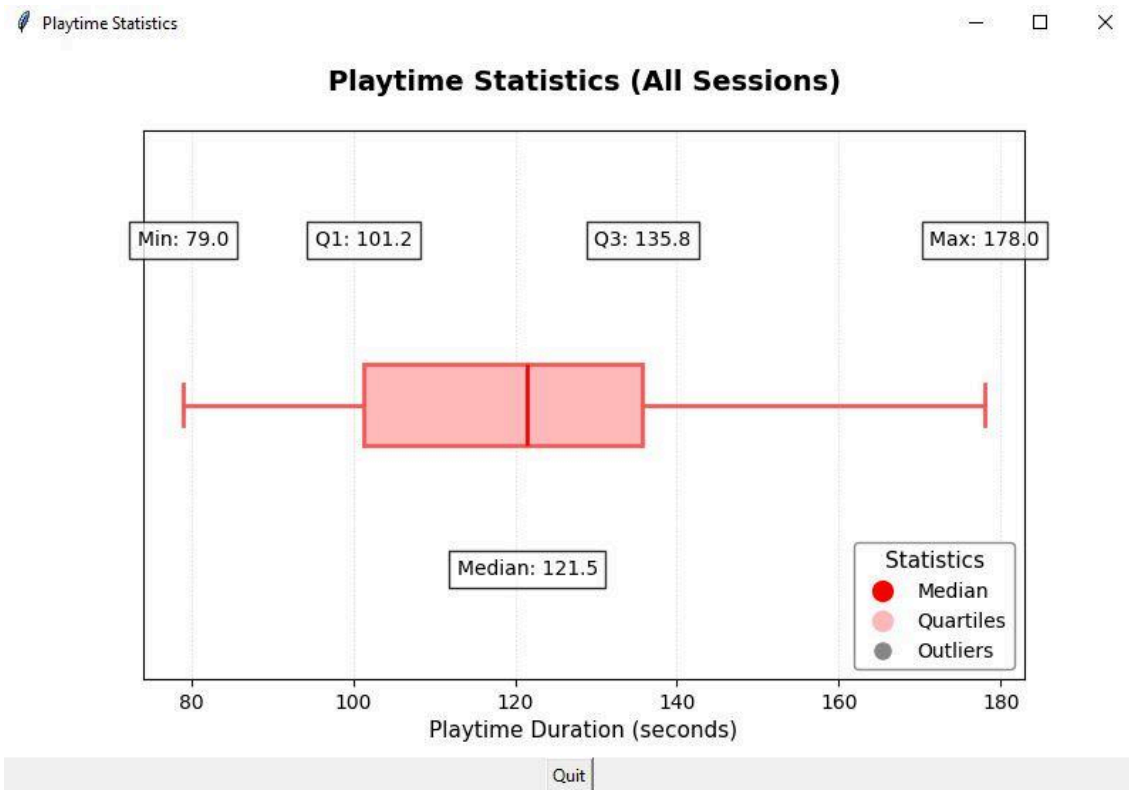
Table Statistical Value

- Feature Name: Time Played, Average Player Dash per boss, Average Ability Usage per character, Enemies Defeated, Most Chosen Character by Player
- Statistical Value:
 - Time Played: Mean, Median, Quartiles, Range
 - Average Player Dash: Percentage Difference
 - Average Ability Usage: Mean, Relative Frequency
 - Enemies Defeated: Total sum of boss defeated per character, Win rate Comparison
 - Most Chosen Character by Player: Percentage Breakdown of character selection, Mode

Graph

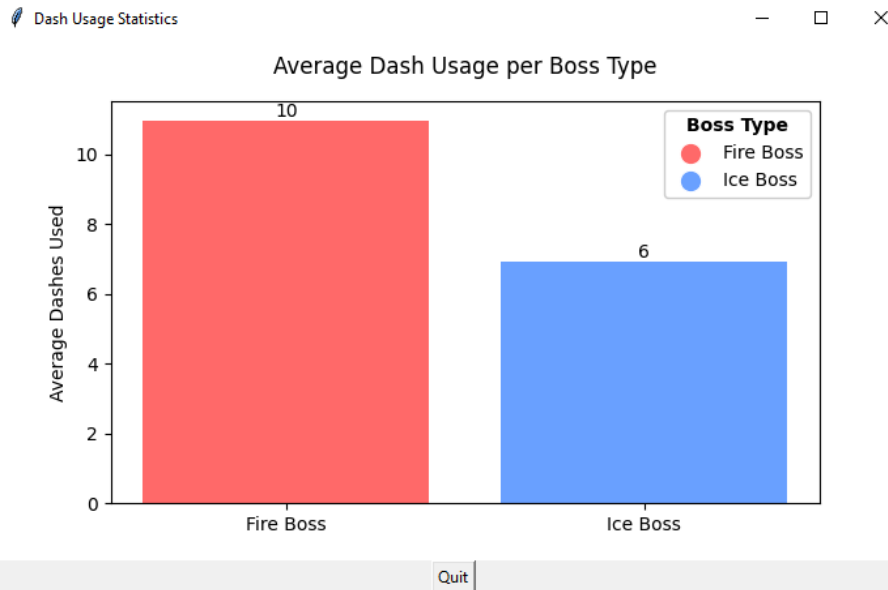
Graph 1: Time Played

-Graph Type: Boxplot



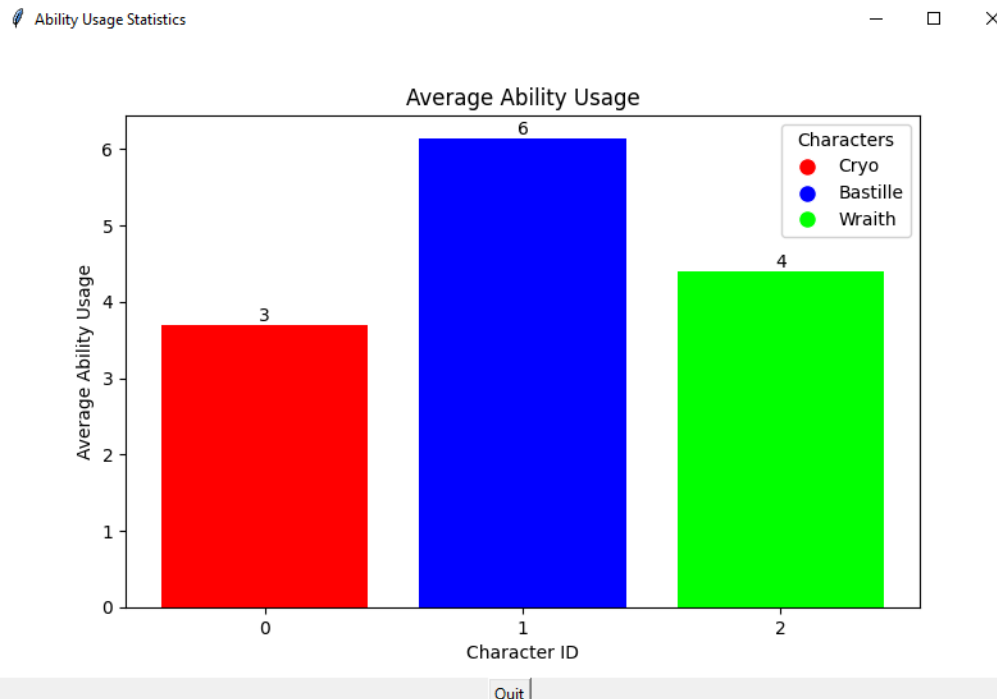
Graph 2: Average Player Dash

- Graph Type: Bar chart
- X-axis: Type of Boss
- Y-axis: Average dash used



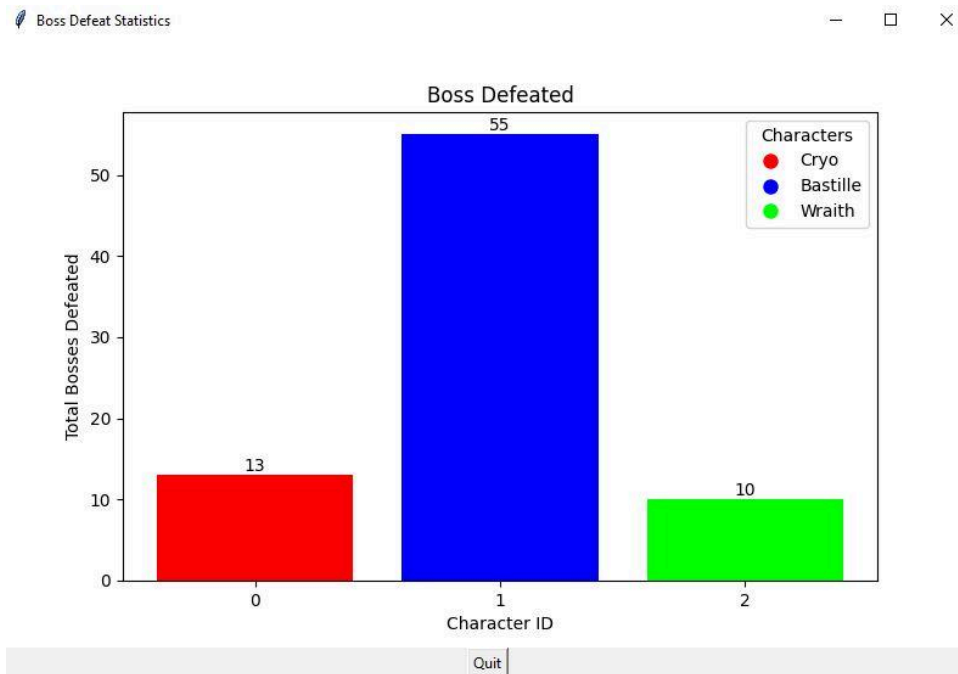
Graph 3: Average Ability Usage

- Graph Type: Bar chart
- X-axis: Character Type
- Y-axis: Average Ability Usage Count



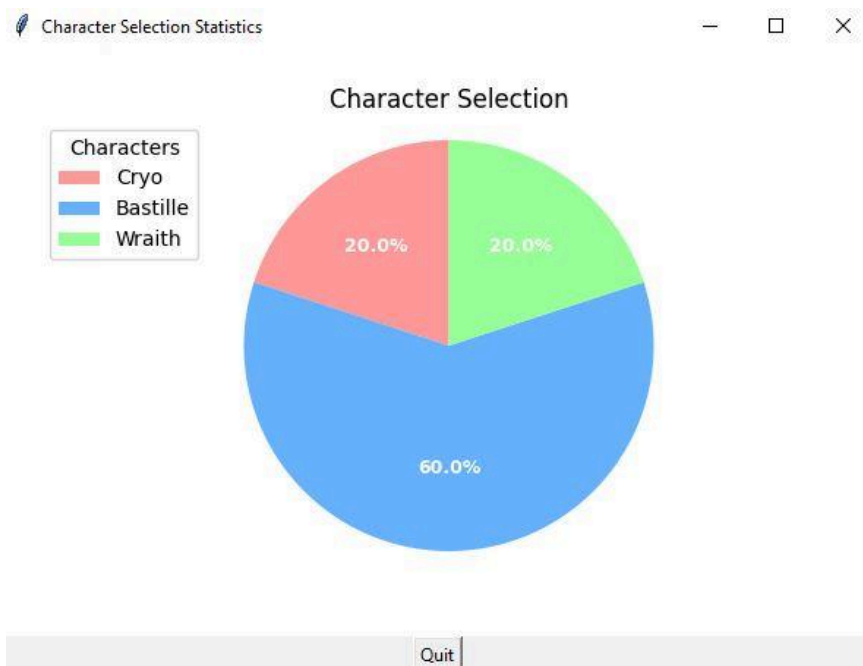
Graph 4: Enemies Defeated

- Graph Type: Bar chart
- X-axis: Character Type
- Y-axis: Boss defeat



Graph 5:

- Graph Type: pie chart



Plan

- 26 March-2 April : Coding game session
- 3 April-9 April: Coding game session
- 10 April-16 April: Coding data session
- 17 April-23 April: : Coding data session
- 24 April-11 May : Coding data session

4. Project Timeline

Week	Task
1 (10 March)	Proposal submission / Project initiation
2 (17 March)	Full proposal submission
3 (24 March)	Designing about character and arena
4 (31 March)	Coding in game
5 (7 April)	Coding in game
6 (14 April)	Submission week (Draft)

5. Document version

Version: 5.0

Date: 11 May 2025

Date	Name	Description of Revision, Feedback, Comments
15/3	Rattapoom	Very good! But don't forget to add project timeline.
16/3	Parima	The proposal is great in detail. Good Work 🌟
29/3	Rattapoom	Depending on the length of your game, playing the game 50 times to collect all the data might not be feasible because it'd take a lot of time. I suggest something like "Number of dashes per minute"

		Also don't forget to put the variables you're planning to use to collect data in the UML class diagram.
29/3	Parima	The diagram needs some revision.